

Технологично училище "Електронни
системи" към Технически университет –
София

Курсова Работа № 2
Операционни Системи
Вариант 3: Състезателна бесеница

Калоян Тодоров № 15

1. Въведение

Създадена е игра "Състезателна бесеница" по протокол TCP. Решението се състои от два файла: `hangman-server` (сървър) и `hangman-client` (клиент). Логиката на самата игра (управление на думата, познаване на букви, проверка за разкриване) се намира в предоставената библиотека `game.c/game.h` и не е променяна.

Двама играчи се свързват към общ сървър. Всеки от тях избира дума, която другия трябва да познае буква по буква. Двамата играят едновременно. Когато и двамата приключат, сървърът сравнява броя на грешни опити и казва резултата: печели играчът с по-малко грешки, а при равен брой грешки резултатът е равенство.

Целта на разработката е да демонстрира работа с TCP сокети, многонишково програмиране с POSIX threads (pthreads) и синхронизация между нишки чрез mutex и условна променлива.

2. Обща архитектура

Системата следва класически модел клиент-сървър. Сървърът е централната точка, която приема връзки, съхранява състоянието на двете думи и взема решение за изхода. Клиентите само показват състоянието на потребителя и препредават въведените букви.

- Сървърът е многонишков - главната нишка приема връзките, а след това за всеки от двамата играчи се създава отделна работна нишка (pthread), която води неговата игра.
- Клиентът е еднонишков: той върви в един последователен цикъл - получаване на състояние, показване, четене на буква, изпращане.
- Комуникацията е текстова, ред по ред (всяко съобщение завършва със знак за нов ред).

Важна особеност е кръстосаното разпределение на думите и защитата срещу измама: думата, която играч подава, никога не се връща обратно към него - тя отива при опонента. Така нито един играч не вижда думата, която сам трябва да познава.

3. Комуникационен протокол

3.1. Основни принципи

Протоколът е проектиран да бъде прост, текстов и лесен за отстраняване на грешки. Той се ръководи от следните правила:

- Всяко съобщение е един текстов ред, който завършва със символ за нов ред `"\n"`.
- Съобщенията започват с ключова дума (WORD, WRONG, OK, ERR, FINAL, YOUR, OPP, SOLVED), което позволява на приемащата страна лесно да разпознае типа на съобщението.

- Символът за връщане на каретка "\r" се игнорира при четене, за да е съвместим протоколът с различни операционни системи.
- Четенето става байт по байт до достигане на "\n" - така една операция връща един логически ред, независимо как TCP е фрагментирал данните.

Цялата комуникация преминава през три последователни фази: Handshake (установяване), Игрови цикъл и Краен резултат.

3.2. Фаза 1 - Handshake - установяване на връзката

Веднага след като TCP връзката е установена, клиентът изпраща думата, която е избрал за другия играч. Сървърът приема думата, проверява я чрез функцията `secret_word_init_from_c_string()` от библиотеката и отговаря. Ако думата е валидна, сървърът изпраща OK; ако не е, изпраща ERR с кратко описание и затваря връзката.

Комуникация	Съобщение	Описание
Клиент -> Сървър	<дума>\n	Думата, която опонентът ще отгатва
Сървър -> Клиент	OK\n	Думата е приета; играта може да започне
Сървър -> Клиент	ERR <причина>\n	Думата е отхвърлена; връзката се затваря

Сървърът изпраща OK на двамата играчи едва след като и ДВЕТЕ думи са получени и валидирани. Така играта стартира синхронно за двамата.

3.3. Фаза 2 - Цикъл на игра

След OK започва играта. За всяка следваща стъпка сървърът първо изпраща актуалното състояние на думата на играча с 2 реда: WORD (скрита дума) и WRONG (грешните букви). След това клиентът изпраща една буква. Сървърът я обработва и отново изпраща обновеното състояние. Този обмен се повтаря, докато думата не бъде позната напълно.

Комуникация	Съобщение	Описание
Сървър -> Клиент	WORD <маска>\n	Маскираната дума: скритите букви са "_", а познатите са разкрити
Сървър -> Клиент	WRONG <букви>\n	Грешните букви, разделени със запетая и интервал
Клиент -> Сървър	<буква>\n	Буквата, която играчът познава
Сървър -> Клиент	SOLVED\n	Изпраща се еднократно, когато думата е разкрита изцяло.

Когато в маската вече няма нито един символ "_", думата е позната и сървърът добавя реда SOLVED.

3.4. Фаза 3 - Краен резултат

Когато играч познае своята дума, неговата нишка изчаква другия играч. Едва след като и двамата играчи са приключили, сървърът изпраща крайния резултат на всеки от тях с три отделни реда: резултат, грешки на играча и на опонента му.

Комуникация	Съобщение	Описание
Сървър -> Клиент	FINAL WIN\n	Играчът е спечелил
Сървър -> Клиент	FINAL LOSE\n	Играчът е загубил
Сървър -> Клиент	FINAL TIE\n	Равенство
Сървър -> Клиент	YOUR <букви>\n	Грешните букви на играча
Сървър -> Клиент	OPP <букви>\n	Грешните букви на опонента.

Резултатът се определя чрез сравнение на `secret_word_incorrect_guess_count()` за двете думи. Сравнението е симетрично, следователно ако единият играч получи WIN, другият задължително получава LOSE, а при равенство и двамата получават TIE.

4. Работа на сървъра (hangman-server)

Сървърът се стартира с номер на порт: `./hangman-server <port>`. Работата му се разделя на инициализация, приемане на връзки и многонишково водене на играта.

4.1. Споделено състояние

Цялото състояние на играта е събрано в една структура (`game_t`), достъпна и от двете работни нишки. Тя съдържа:

- `words[2]` - двете думи (`secret_word_t`); `words[i]` е думата, която играч `i` отгатва.
- `fds[2]` - сокетите (файлови дескриптори) на двамата приети клиента.
- `done[2]` - флагове за завършване; `done[i]=1` означава, че играч `i` е познал думата си.
- `lock` - mutex за взаимно изключване при достъп до споделените данни.
- `cond` - условна променлива за изчакване, докато и двамата приключат.

4.2. Инициализация

- Игнорира сигнала SIGPIPE (`signal(SIGPIPE, SIG_IGN)`), за да не "падне" процесът, ако пише в прекъсната връзка
- Създава TCP сокет с `socket(AF_INET, SOCK_STREAM, 0)`
- Задава опцията `SO_REUSEADDR`, която позволява веднага повторно ползване на порта след рестарт на сървъра.

- Свързва сокета към 0.0.0.0:<port> с bind() и започва да слуша с listen().
- Пуска в терминала "Listening on <port>..." и нулира done[0] и done[1].

4.3. Приемане на двамата играчи

В цикъл сървърът извиква accept() два пъти. За всеки приет клиент веднага чете първия ред - думата, която този играч подава (запазена в raw[i]). Ако четенето се провали, слотът се освобождава и се прави нов опит за този играч.

След като и двете думи са получени, сървърът затваря слушащия сокет (повече връзки не са нужни) и извършва кръстосаното разпределение:

```
words[0] = raw[1]    // играч 1 познава думата на играч 2
words[1] = raw[0]    // играч 2 познава думата на играч 1
```

Всяка дума се инициализира с secret_word_init_from_c_string(). Ако някоя дума е невалидна, съответният играч получава "ERR invalid word" и играта не започва.

4.4. Стартиране на нишките

При успех сървърът изпраща ОК на двамата и създава две работни нишки с pthread_create(), по една за всеки играч (player_thread). Главната нишка изчаква двете нишки с pthread_join(), след което освобождава ресурсите (думите, mutex-а и условната променлива).

4.5. Работна нишка на играч (player_thread)

Всяка нишка обслужва един играч и преминава през следните стъпки:

- Изпраща началното състояние - WORD (изцяло скрита дума) и WRONG (празно).
- Влиза в цикъл, който продължава, докато secret_word_is_solved() не е изпълнено: чете една буква от клиента, подава я на secret_word_guess() и изпраща обновените WORD и WRONG.
- Когато думата е позната, изпраща SOLVED.
- Влиза в режим на синхронизация: отбелязва done[id]=1, известява другата нишка и изчаква, докато и двамата приключат.
- Изчислява резултата чрез сравнение на броя грешки и изпраща FINAL, YOUR и OPP.
- Затваря сокета на клиента.

4.6. Помощни функции в сървъра

- net_readline() - чете от сокета байт по байт до "\n" и връща готов низ; "\r" се пропуска. Връща -1 при прекъсната връзка.
- net_sendline() - форматира съобщение (като printf), добавя "\n" и го изпраща с send() и флаг MSG_NOSIGNAL (за да не генерира SIGPIPE).
- fmt_masked() - изгражда думата ("d_g") от secret_word_letter_at().
- fmt_wrong() - изброява грешните букви, разделени със запетая и интервал.

5. Работа на клиента (hangman-client)

Клиентът се стартира с три аргумента: `./hangman-client <host> <port> <дума>`", където `<дума>` е думата, която другия играч още трябва да познава. Клиентът е изцяло еднонишков.

5.1. Свързване

- Игнорира SIGPIPE, за да е устойчив при внезапно прекъсване.
- Изключва буферирането на stdout (setvbuf с `_IONBF`), така че всеки изведен ред да достига веднага до терминала или до тестовата система, без забавяне.
- Създава сокет, преобразува текстовия адрес с `inet_pton()` и се свързва със сървъра чрез `connect()`.

5.2. Фаза 1 в клиента

Клиентът изпраща своята дума и зачаква отговор. Ако отговорът не започва с "OK", клиентът отпечатва съобщението и приключва с код за грешка.

5.3. Фаза 2 в клиента - игрови цикъл

В безкраен цикъл клиентът изпълнява следната последователност:

- Чете реда WORD и запазва маската; чете реда WRONG и запазва грешните букви.
- Извежда на екрана два реда: "Word: <маска>" и "Incorrect guesses: <букви>".
- Проверява маската: ако в нея вече няма символ "_", думата е позната и цикълът се прекъсва (преминава към фаза 3).
- В противен случай чете един ред от стандартния вход (stdin), взема първия символ като буква-опит и го изпраща на сървъра.

Ако четенето от сървъра или от stdin се провали (край на входа или прекъсната връзка), клиентът преминава директно към фазата на резултата.

5.4. Фаза 3 в клиента - резултат

Клиентът чете редове, докато не получи и трите очаквани съобщения - FINAL, YOUR и OPP. За проследяване кои са вече получени се използва побитова маска (got): бит 1 за FINAL, бит 2 за YOUR, бит 4 за OPP. Цикълът приключва, когато got = 7 (всички три са получени). Евентуални други редове (например SOLVED) се пропускат тихо.

Накрая клиентът превежда вердикта в съобщение към потребителя:

- WIN -> "YOU WIN! :)"
- LOSE -> "You Lose! :("
- TIE -> "Tie :/"

След резултата се извеждат собствените неверни букви и тези на опонента.

5.5. Помощни функции в клиента

- `net_readline()` - аналогична на сървърната: чете един ред от сокета байт по байт.

- `net_sendline()` - добавя "\n" към низа и го изпраща с флаг `MSG_NOSIGNAL`.

5.5. Обработка на грешни въвеждания (edge cases)

Преди да изпрати буква на сървъра, клиентът я проверява и при нужда показва съобщение и пита отново, без да губи ход и без да изпраща нищо. Покрити са два случая:

- Въведена е цяла дума или повече от един символ - например потребителят напише "dog" вместо една буква. Клиентът показва "Please enter only one letter." и изчаква ново въвеждане.
- Буквата вече е била побвана - проверява се дали символът се среща сред познатите букви или сред грешните. Ако да, клиентът показва "Already guessed 'x', try again." и изчаква ново въвеждане.

6. Код

6.1. Структура и помощни функции (сървър)

Цялото състояние на играта се събира в една структура, видима за двете нишки. Ключовата дума `volatile` при `done` подсказва на компилатора, че стойността може да бъде променена от друга нишка и не бива да се кешира в регистър.

```
typedef struct {
    secret_word_t words[2];    // words[i] = думата, която
    играч i отгатва
    int fds[2];                // сокетите на двамата клиента
    volatile int done[2];      // done[i]=1, когато играч i е
    приключил
    pthread_mutex_t lock;      // взаимно изключване
    pthread_cond_t cond;       // условна променлива за изчакване
} game_t;

static game_t G;              // единствен глобален
екземпляр
```

Функцията `net_readline()` чете един логически ред. Тя взема по един байт с `recv()` и спира при "\n". Така е без значение как TCP е фрагментирал потока, винаги се връща цял ред. Символът "\r" се пропуска за съвместимост между ОС.

```
static int net_readline(int fd, char *buf, size_t cap) {
    size_t i = 0;
    for (;;) {
        char c;
        ssize_t n = recv(fd, &c, 1, 0);    // чете 1 байт
        if (n <= 0) return -1;              // връзката е
        затворена
        buf[i++] = c;
    }
}
```

```

        if (c == '\n') break;                // край на реда
        if (c != '\r' && i < cap - 1) buf[i++] = c;
    }
    buf[i] = '\0';
    return (int)i;
}

```

Функцията `net_sendline()` приема форматен низ и аргументи, добавя "\n" накрая и изпраща реда. Флагът `MSG_NOSIGNAL` предотвратява генерирането на сигнал `SIGPIPE`, ако отсрещната страна е затворила връзката.

```

static void net_sendline(int fd, const char *fmt, ...) {
    char buf[BUF];
    va_list ap; va_start(ap, fmt);
    int n = vsnprintf(buf, sizeof(buf) - 2, fmt, ap);
    va_end(ap);
    if (n <= 0 || n >= (int)sizeof(buf) - 1) return;
    buf[n] = '\n'; buf[n+1] = '\0';
    send(fd, buf, (size_t)(n + 1), MSG_NOSIGNAL);
}

```

6.2. Изграждане на маската и неверните букви

`fmt_masked()` обхожда всяка позиция от думата. Ако буквата е разкрита (`SECRET_WORD_LETTER_REVEALED`), записва самата буква, а ако не е все още - записва "_".

```

static void fmt_masked(const secret_word_t *w, char *out,
size_t cap) {
    size_t j = 0;
    for (size_t i = 0; i < w->word_length && j+1 < cap; i++)
    {
        char c = '_';
        if (secret_word_letter_at(w, i, &c) ==
SECRET_WORD_LETTER_REVEALED)
            out[j++] = c;                // разкрита буква
        else
            out[j++] = '_';              // все още скрита
    }
    out[j] = '\0';
}

```

6.3. Приемане на играчите и кръстосано разпределение

Главната нишка приема двама клиента. За всеки веднага прочита подадената дума в `raw[i]`. Ако четенето се провали, слотът се пробва отново (`i` не се увеличава).

```
char raw[2][MAX_WORD];
for (int i = 0; i < 2; ) {
    G.fds[i] = accept(srv, NULL, NULL);
    if (G.fds[i] < 0) { perror("accept"); return 1; }
    if (net_readline(G.fds[i], raw[i], sizeof raw[i]) < 0) {
        net_sendline(G.fds[i], "ERR connection error");
        close(G.fds[i]);
        continue;          // повтаря опита за този слот
    }
    i++;
}
```

Следва кръстосаното разпределение и валидирането: думата на единия играч става дума за познаване на другия. При невалидна дума се изпраща ERR.

```
// words[0] = raw[1] (играч 0 отгатва думата на играч 1)
// words[1] = raw[0] (играч 1 отгатва думата на играч 0)
if (!secret_word_init_from_c_string(&G.words[0], raw[1])) {
    net_sendline(G.fds[1], "ERR invalid word");
    close(G.fds[0]); close(G.fds[1]); return 0;
}
if (!secret_word_init_from_c_string(&G.words[1], raw[0])) {
    net_sendline(G.fds[0], "ERR invalid word"); ...
}
net_sendline(G.fds[0], "OK");
net_sendline(G.fds[1], "OK");
```

6.4. Работната нишка (player_thread)

Тялото на нишката изпраща началното състояние, после върти цикъл на отгатване, докато думата не е позната, и накрая изпраща SOLVED.

```
fmt_masked(word, masked, sizeof masked);
fmt_wrong (word, wrong,  sizeof wrong);
net_sendline(fd, "WORD %s",  masked);
net_sendline(fd, "WRONG %s", wrong);

while (!secret_word_is_solved(word)) {
    if (net_readline(fd, line, sizeof line) < 0) goto done;
```

```

    char guess = line[0];
    secret_word_guess(word, guess);          // обработка буквата
    fmt_masked(word, masked, sizeof masked);
    fmt_wrong (word, wrong,  sizeof wrong);
    net_sendline(fd, "WORD %s",  masked);
    net_sendline(fd, "WRONG %s", wrong);
}
net_sendline(fd, "SOLVED");

```

Резултатът се определя чрез сравнение на броя грешни опити за двете думи:

```

int my_err  =
secret_word_incorrect_guess_count(&G.words[id]);

int opp_err = secret_word_incorrect_guess_count(&G.words[1 -
id]);

const char *verdict =

    (my_err < opp_err) ? "WIN"  :

    (my_err > opp_err) ? "LOSE" : "TIE";

net_sendline(fd, "FINAL %s", verdict);
net_sendline(fd, "YOUR %s",  my_wrong);
net_sendline(fd, "OPP %s",   opp_wrong);

```

6.5. Игрови цикъл на клиента

Клиентът чете двата реда WORD и WRONG, показва състоянието и ако в маската няма "_" прекъсва цикъла. Иначе чете буква от stdin и я изпраща.

```

for (;;) {
    if (net_readline(line, sizeof line) < 0) goto result;
    if (strncmp(line, "WORD ", 5) == 0)
        strncpy(masked, line + 5, sizeof masked - 1);
    if (net_readline(line, sizeof line) < 0) goto result;
    if (strncmp(line, "WRONG ", 6) == 0)
        strncpy(wrong, line + 6, sizeof wrong - 1);

    printf("Word: %s\n", masked);
    printf("Incorrect guesses: %s\n", wrong);

    if (strchr(masked, '_') == NULL) break;    // думата е
    позната

    char inp[BUF];

```

```

        if (fgets(inp, sizeof inp, stdin) == NULL) goto result;
        char guess[3] = { inp[0], '\0' };
        net_sendline(guess);
    }

```

Във фаза 3 клиентът чете редове, докато не събере и трите съобщения. Побитовата маска got се запълва бит по бит и цикълът спира при стойност 7 (1, 2, 4).

```

int got = 0; // бит1=FINAL, бит2=YOUR, бит4=OPP
while (got != 7) {
    if (net_readline(line, sizeof line) < 0) break;
    if (strncmp(line, "FINAL ", 6) == 0) { ...; got |= 1; }
    else if (strncmp(line, "YOUR ", 5) == 0) { ...; got |= 2; }
    else if (strncmp(line, "OPP ", 4) == 0) { ...; got |= 4; }
}

```

7. Синхронизация на нишките

Двете нишки на сървъра работят независимо по време на познаването, но крайният резултат не може да се изпрати, преди и двамата играчи да са приключили. За това се използват mutex (G.lock) и условна променлива (G.cond) върху флаговете done[2].

Когато нишка i завърши цикъла на играта и изпрати SOLVED, тя изпълнява следното:

- Заклучва mutex-а с pthread_mutex_lock().
- Слага done[i] = 1.
- Известява другата нишка с pthread_cond_broadcast().
- В цикъл изчаква с pthread_cond_wait(), докато условието (done[0] && done[1]) е изпълнено.
- Отключва mutex-а и продължава към изпращането на резултатс.

В код тази последователност изглежда така:

```

pthread_mutex_lock(&G.lock);
G.done[id] = 1; // отбелязва приключване
pthread_cond_broadcast(&G.cond); // събужда другата нишка
while (!G.done[0] || !G.done[1]) // изчаква и двамата
    pthread_cond_wait(&G.cond, &G.lock);
pthread_mutex_unlock(&G.lock);

```

Тази схема гарантира коректност без активно изчакване (busy-waiting): нишката, която приключи първа, "заспива" и не натоварва процесора, докато не бъде събудена от втората нишка. Достъпът до общите данни винаги е защитен от mutex-а, което изключва състезание за ресурси (race conditions).

8. Компиляция и стартиране

8.1. Компиляция

В основната директория на проекта:

```
make all
```

Създават се двата изпълними файла `hangman-server` и `hangman-client`. Сървърът се свързва с библиотеката `pthread` чрез флага `-lpthread`.

8.2. Стартиране

Сървър (например на порт 9000):

```
(kali㉿kali)-[~/Desktop/os_hangman]
$ ./hangman-server 9000
Listening on 9000 ...
```

Два клиента в отделни терминали (думите трябва да са на латиница):

Играч 1:

```
(kali㉿kali)-[~/Desktop/os_hangman]
$ ./hangman-client 127.0.0.1 9000 cat
```

Играч 2:

```
(kali㉿kali)-[~/Desktop/os_hangman]
$ ./hangman-client 127.0.0.1 9000 dog
```

9.3. Примерна игра

Играч 1:

```
(kali㉿kali)-[~/Desktop/os_hangman]
$ ./hangman-client 127.0.0.1 9000 cat
Word: __
Incorrect guesses:
d
Word: d__
Incorrect guesses:
f
Word: d__
Incorrect guesses: f
i
Word: d__
Incorrect guesses: f, i
h
Word: d__
Incorrect guesses: f, h, i
o
Word: do_
Incorrect guesses: f, h, i
g
Word: dog
Incorrect guesses: f, h, i
You Lose! :(
Your incorrect guesses: f, h, i
Opponent's incorrect guesses: j
```

Игрок 2:

```
(kali@kali)-[~/Desktop/os_hangman]
$ ./hangman-client 127.0.0.1 9000 dog
Word: ____
Incorrect guesses:
j
Word: ____
Incorrect guesses: j
c
Word: c__
Incorrect guesses: j
a
Word: ca_
Incorrect guesses: j
t
Word: cat
Incorrect guesses: j
YOU WIN! :)
Your incorrect guesses: j
Opponent's incorrect guesses: f, h, i
(kali@kali)-[~/Desktop/os_hangman]
```